

Win API – Основы

Прежде всего, следует отметить, что на чистом API в настоящее время практически никто не разрабатывает программное обеспечение, поскольку это нецелесообразно. Код, созданный с использованием только API, не является объектно-ориентированным, имеет значительный объем и требует ручного написания каждой мелочи. Разработка с использованием API сопряжена с необходимостью бесконечного количества циклов обработки сообщений для каждого окна, что увеличивает вероятность возникновения ошибок и усложняет их диагностику. Для иллюстрации данной проблемы представляется несколько программ, которые демонстрируют работу с API-функциями, циклами обработки сообщений и ресурсами. Эти аспекты будут полезны в дальнейшем.

В определенной степени библиотека MFC (Microsoft Foundation Class) позволяет скрыть от программиста сложные циклы и функции API. Тем не менее, в ситуациях, когда требуется доступ к функционалу, не описанному в стандартных классах, API оказывается незаменимым. Актуальные знания API-кода необходимы, даже на минимальном уровне, для лучшего понимания структуры MFC и других библиотек классов. Важно отметить, что вызовы API-функций возможны даже в среде C++ Builder, хотя необходимость в этом возникает крайне редко. Разработчики Windows, вероятно, переоценили возможность программистов обходиться единственным библиотечным файлом windows.h. Изначально ситуация выглядела приемлемо, однако со временем библиотека значительно разрослась. С каждой новой версией Windows количество API-функций продолжало увеличиваться. Например, файл windows.h содержит более 1000 констант. Например:

```
#define WM_QUIT 0x0012
```

А также прототипы VCEX API-функций. Что было делать? Современный файл windows.h разделили на 3 составляющих: **kernel**, **user** и **gdi**. Теперь они носят названия:

- winuser.h - объявления из USER.EXE или ассоциированные с ним
- wingdi.h - интерфейс графических устройств
- windef.h - определение фундаментальных типов данных
- winnls.h - поддержка национального языка в локализованных версиях
- wincon.h - консольный ввод-вывод, минуя интерфейс графических устройств
- winbase.h - базовый программный интерфейс.
- ...

В большинстве случаев к API-программе придётся подключать только windows.h.

Перейдем к созданию проекта с использованием Win API.

Создание пустого проекта C++

В среде Visual Studio создайте новый пустой проект на языке C++.

Настройка проекта

После создания проекта вам будет доступна его структура. Теперь необходимо ввести текст программы и добавить его в проект для корректной работы.

Особенности работы с пустыми проектами

Важно отметить, что в пустом проекте все элементы, такие как исходные файлы с расширением .cpp и библиотеки с расширением .h, необходимо добавлять вручную.

В левой части экрана находится окно файлов Вашего проекта, которое в данный момент пустое. В этом окне структура организована в условные папки, названия которых зависят от типа содержимого: **Source** (для файлов с расширением .cpp), **Header** (для заголовочных файлов с расширением .h) и **Resource** (для файлов ресурсов с расширением .rc). При добавлении различных файлов в проект Вы сможете наблюдать их расположение в соответствующих папках.

Для создания нового файла нажмите правой кнопкой мыши по папке **Source**, далее Add -> New Item -> .cpp. Сохраните его под названием First.cpp, а затем добавьте его в проект.

Теперь на экране откроется файл First.cpp, куда Вы можете вводить исходный текст программы. Убедитесь, что файл появился в соответствующей папке Source. Теперь всё готово для написания Вашей первой программы.

Первая программа на Win API

Вашей первой программой будет следующий код:

```
#include<windows.h>

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpszCmdLine, int
nCmdShow)
{
    MessageBox(0, "Привет, Win API", "Первое окно...", MB_ICONINFORMATION|MB_OK);
}
```

Ввод кода

Введите указанный текст в файл First.cpp.

Запуск программы

Для компиляции и запуска программы нажмите клавишу F5. На этом этапе компилятор запросит у Вас подтверждение на создание исполняемого файла.

Результат

При корректном вводе кода на экране появится окно с информационным сообщением.

Создание исполняемого файла

После выполнения программы в папке проекта будет создана вложенная папка Debug, в которую будет помещён исполняемый файл с именем First.exe.

Таким образом, Вы успешно создали и запустили Ваше первое приложение, использующее Win API.

Эта программа не содержит главного окна, в отличие от большинства Windows программ. По сути, это пустое тело функции WinMain(), в котором просто выдаётся информационное сообщение.

API функция MessageBox широко применяется в Windows для выдачи всевозможных сообщений:

информационных (MB_ICONINFORMATION),
вопросов (MB_ICONQUESTION),
сообщений об ошибках (MB_ICONSTOP),
предупреждений (MB_ICONEXCLAMATION).

Число кнопок также может варьироваться: "ОК" (MB_OK), "Отмена" (MB_CANCEL), "Пропустить" (MB_IGNORE). Это так называемое стандартное диалоговое окно.

Создание нового проекта

Завершите работу с текущим проектом.

Создайте новый пустой проект, например, назовите его First1.

Добавьте в проект пустой одноимённый файл First1.cpp, как это было сделано в предыдущей программе.

Теперь введите код, который соответствует стандартам Win API. Программа создаст окно, в котором пользователь увидит приветственное сообщение.

Внимание! Если ранее не было опыта работы с Win API, представленный ниже код может показаться сложным. Данный код является минимальной программой для создания окна Windows. Окно будет иметь однодокументный интерфейс (SDI - Single Document Interface) и содержать надпись: "Hello, Win API!".

Как и любое другое окно Windows, оно может быть свернуто, развернуто и закрыто. При наведении курсора на кнопки будут отображаться всплывающие подсказки. Окно также имеет системное меню, и его размер можно изменять с помощью мыши.

Вывод текста в окно осуществляется через сообщение WM_PAINT, поэтому сразу рассматривается цикл обработки сообщений окна, который уже знаком в теории.

Эта программа будет служить базовой заготовкой для последующих проектов, в которых будет добавляться новый функционал, что избавит от необходимости вводить код каждый раз заново. Таким образом, останется лишь модифицировать существующий код в соответствии с потребностями.

```
#include<windows.h>

//Создаём прототип функции окна
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
//объявляем имя программы
char szProgName[]="Имя программы";

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpszCmdLine, int nCmdShow)
{
    HWND hWnd; //идентификатор окна
    MSG lpMsg;
```

```

WNDCLASS w; //создаём экземпляр структуры WNDCLASS и начинаем её заполнять

w.lpszClassName=szProgName; //имя программы
w.hInstance=hInstance; //идентификатор текущего приложения
w.lpfnWndProc=WndProc; //указатель на функцию окна
w.hCursor=LoadCursor(NULL, IDC_ARROW); //загружаем курсор в виде стрелки
w.hIcon=0; //иконки у нас не будет пока
w.lpszMenuName=0; //и меню пока не будет
w.hbrBackground=(HBRUSH)GetStockObject(WHITE_BRUSH); //цвет фона окна - белый
w.style=CS_HREDRAW|CS_VREDRAW; //стиль окна - перерисовываемое по x и по y
w.cbClsExtra=0;
w.cbWndExtra=0;

//Если не удалось зарегистрировать класс окна - выходим
if(!RegisterClass(&w))
return 0;

//Создадим окно в памяти, заполнив аргументы CreateWindow

hWnd=CreateWindow(szProgName, //Имя программы
"Моя первая программа!", //Заголовок окна
WS_OVERLAPPEDWINDOW, //Стиль окна - перекрывающееся
100, //положение окна на экране по x
100, //по y
500, //размеры по x
400, //по y
(HWND)NULL, //идентификатор родительского окна
(HMENU)NULL, //идентификатор меню
(HINSTANCE)hInstance, //идентификатор экземпляра программы
(HINSTANCE)NULL); //отсутствие дополнительных параметров

//Выводим окно из памяти на экран
ShowWindow(hWnd, nCmdShow);
//Обновим содержимое окна
UpdateWindow(hWnd);

//Цикл обработки сообщений

while(GetMessage(&lpMsg, NULL, 0, 0)) { //Получаем сообщение из очереди
TranslateMessage(&lpMsg); //Преобразуем сообщения клавиш в символы
DispatchMessage(&lpMsg); //Передаём сообщение соответствующей функции окна
}
return(lpMsg.wParam);
}

//Функция окна
LRESULT CALLBACK WndProc(HWND hWnd, UINT messg,
WPARAM wParam, LPARAM lParam)
{
HDC hdc; //создаём контекст устройства
PAINTSTRUCT ps; //создаём экземпляр структуры графического вывода

//Цикл обработки сообщений
switch(messg)
{
//сообщение рисования
case WM_PAINT :
//начинаем рисовать
hdc=BeginPaint(hWnd, &ps);
//здесь вы обычно вставляете свой текст:
TextOut(hdc, 150,150, "Hello, WIN API!", 26);
//закругляемся
//обновляем окно

```

```

ValidateRect(hWnd, NULL);
//заканчиваем рисовать
EndPaint(hWnd, &ps);
break;

//сообщение выхода - разрушение окна
case WM_DESTROY:
PostQuitMessage(0); //Посылаем сообщение выхода с кодом 0 - нормальное завершение
break;

default:
return(DefWindowProc(hWnd, messg, wParam, lParam)); //освобождаем очередь приложения от
нераспознанных
}
return 0;
}

```

Данная программа представляет собой готовый шаблон для приложений API, который не может быть значительно сокращён. Тем не менее, добавляя лишь небольшие фрагменты кода, можно достигать впечатляющих результатов.

Теперь следует более подробно рассмотреть структуру этой программы.

1. В функции WinMain создаются экземпляры структур: окна HWND, сообщений MSG и класса окна WNDCLASS.

2. Заполняется структура WNDCLASS. Каждое поле прокомментировано в листинге.

3. Регистрируется класс окна функцией RegisterClass, передавая, как аргумент, заполненную структуру. Как правило, класс нельзя зарегистрировать если не хватает памяти. Тогда приложение будет закрыто return 0;

4. Далее запускается цикл обработки сообщений. Тело цикла находится в WndProc.

5. Дальше идёт тело WndProc, в котором создаётся экземпляр контекста устройства (HDC) и начинает заполнять контекст. Как Вы помните, контекст содержит выбранные Вами шрифты, перья, кисти, а также всё, что Вы изволили нарисовать. В данном случае, выведется текст.

6. Экземпляр PAINTSTRUCT используется в BeginPaint / EndPaint.

7. Цикл обработки сообщений. С помощью switch / case, получаете аргумент messg. Если это сообщение рисования WM_PAINT, рисование, если WM_QUIT - выход из программы, если нераспознанное default, освобождение очереди приложения.

8. В ветви WM_PAINT начинается рисование с помощью BeginPaint, а заканчивается EndPaint. Это своего рода операторские скобки, как { }. Между ними Вы можете рисовать. Они выделяют контекст и позволяют работать с ним, а потом закрывают его.

9. TextOut(HDC, int x, int y, char *str, int num) - вывод текста. Первый аргумент - идентификатор контекста, дальше координаты по x и по y, дальше строка, которую собираетесь вывести, дальше - длина строки в символах.

10. Обновление клиентской области окна - ValidateRect.

Как видно, структура программы проста, но содержит множество элементов. Здесь представлены новые названия функций, понятия, типы данных и конструкции. При регулярном вводе кода Вы привыкнете к нему и сможете выделять ключевые моменты, которые интересуют в данный момент. Обычно это касается ветвей обработки сообщений, а также мест, где заполняется структура WNDCLASS и вызывается функция CreateWindow.

Visual Studio включает встроенное средство редактирования иконок. Также Вы можете просто скопировать файл с расширением .ico в папку с проектом — этот файл будет использоваться в качестве Вашей иконки.

Иконки, курсоры и изображения относятся к ресурсам приложения, ссылки на которые находятся в файле с расширением .rc. Этот файл ресурсов не компилируется и предоставляет возможность вносить изменения. В нем содержатся подробные описания диалоговых окон, типы всех элементов управления, а также иконки, курсоры и изображения. Файл ресурсов можно рассматривать как разметку, описывающую расположение и свойства различных элементов.

На текущий момент в проекте нет ресурсов, но сейчас они будут добавлены. Нажмите правой кнопкой мыши по папке Resource, Add -> Resource. Откроется окно для добавления ресурса.

Через это окно можно добавлять изображения, курсоры, диалоговые окна, меню, панели управления и другие ресурсы. Выберите тип ресурса Icon и нажмите кнопку New, если хотите воспользоваться встроенным редактором, или кнопку Import, чтобы вставить готовый файл-иконку. В любом случае иконка будет открыта в редакторе ресурсов.

После этого в папке Resource Files появится файл с расширением .rc и файл иконки. Если щёлкнуть по файлу .rc, отобразится содержимое этого файла, в данном случае — иконка. Компилятор присвоит ей имя IDI_ICON1 (где IDI является префиксом идентификаторов иконок).

При желании имя можно изменить в строке ID (Properties) можно изменить имя. Рекомендуется заменить безликое `IDI_ICON1` на `IDI_MYICON`, чтобы точно знать, что это ваша иконка. После внесения изменений запустите проект, нажав клавишу F5.

Для корректного отображения добавленной иконки необходимо уведомить окно о её наличии. В противном случае вместо заданной иконки будет отображаться стандартный элемент интерфейса.

Первоначально следует установить связь с иконкой. Для этого необходимо объявить переменную `szIconName`, которая будет содержать имя, присвоенное иконке (например, **`IDI_MYICON`**). Рекомендуется разместить объявление переменной `szIconName` сразу под объявлением имени программы, в начале исходного кода. Это позволит компилятору правильно идентифицировать иконку во время выполнения программы.

```
char szProgName[]="Имя программы"; //эта строка уже была  
char szIconName[]="IDI_MYICON"; //а эту добавляем
```

При заполнении структуры `WNDCLASS` было указано, что иконок не существует (`w.hIcon=0`). Необходимо найти следующую строку кода:

```
w.hIcon=0;
```

Теперь исправьте эту строку:

```
w.hIcon=LoadIcon(hInstance, szIconName);
```

`hInstance` - идентификатор текущего приложения.

Второй способ.

Вы можете сами написать свой файл ресурсов. Создайте текстовый файл в той же папке, что и Ваш проект. Сохраните его, как `First.rc`. Введите в него следующий текст:

```
//First1.rc
```

```
#include "windows.h"
```

```
IDI_MYICON ICON "myicon.ico"
```

У файлов *.rc свой язык. Сначала идет объявление идентификатора (IDI_MYICON), потом указано, что это иконка ICON, потом указано имя файла, который расположен в текущей папке - myicon.ico. Добавьте этот файл RC в проект.

Предупреждение! Иногда компилятор может упорно не включать новую иконку, выдавая программу с стандартными флажками вместо заданных иконок. В таких случаях можно воспользоваться следующими рекомендациями:

Попробуйте заново создать исполняемый файл. Для этого в меню выберите пункт Build, а затем Build Solution или Rebuild. Иногда это помогает решить проблему. Если предыдущий шаг не дал результатов, попробуйте создать файл ресурсов самостоятельно с нуля.

Обратите внимание, что некоторые менее продвинутые компиляторы могут не поддерживать иконки размером 32x32 пикселей и требуют иконки размером 16x16 пикселей.

Настройка одного проекта значительно упростит дальнейшую работу.

Стоит отметить, что подобные сложности возникают преимущественно при работе с API.

Кроме пользовательских иконок, можно вывести стандартные иконки в функции LoadIcon, если ваше окно подходит под их назначение:

IDI_APPLICATION - иконка приложения по умолчанию
IDI_ASTERISK - символ i (информационные приложения)
IDI_EXCLAMATION - восклицательный знак (предупреждение)
IDI_HAND - знак STOP (ошибка)
IDI_QUESTION - вопросительный знак
IDI_WINLOGO (NULL) - флажок

Потренируйтесь, меняя иконки, например:

```
w.hIcon=LoadIcon(hInstance, IDI_APPLICATION);
```

Курсор менять предельно просто. Так же, как и иконку. Снова выберите пункт меню в Resource Add -> Resource, только теперь выберите Cursor. Щёлкнем кнопку New, если хотим нарисовать его сами или Import, если хотим добавить существующий уже файл *.cur

Теперь в Solution Explorer (First.rc) добавился идентификатор IDC_CURSOR1. Поменяйте его имя в окне Properties на IDC_MYCURSOR.

Теперь объявим переменную, которая будет содержать имя курсора.

```
char szProgName[]="Имя программы"; //эта строка уже была
char szIconName[]="IDI_MYICON"; //и эта уже была
char szCursorName[]="IDC_MYCURSOR" //а эту добавьте
```

И в объявлении структуры WNDCLASS тоже поменяйте строку:

```
w.hCursor=LoadCursor(hInstance, szCursorName); //теперь подключен
пользовательский курсор.
```

В некоторых случаях, когда компилятор не воспринимает изменения, возможно ручное написание кода RC-файла. В случае самостоятельного создания, файл будет выглядеть следующим образом:

```
//First1.rc
#include "windows.h"
IDI_MYICON ICON "myicon.ico"
IDC_MYCURSOR "mycursor.cur"
```

Кроме пользовательского курсора, мы в любой момент можем поменять стандартную стрелку на любой другой стандартный курсор:

IDC_CROSS - крестик
IDC_HELP - курсор со знаком вопроса
IDC_APPSTARTING
IDC_ARROW - обычная стрелка
IDC_IBEAM - текстовый курсор I
IDC_NO - перечёркнутый круг
IDC_SIZEALL - четырёхсторонняя стрелка
IDC_SIZENESW - двухсторонняя диагональная стрелка (северо-восток и юго-запад)
IDC_SIZENS - ---//---- (на север и на юг)
IDC_SIZENWSE ---//--- (северо-запад и юго-восток)
IDC_SIZEWE (на запад и на восток)
IDC_UPARROW - вертикальная стрелка
IDC_WAIT - песочные часы

Умело используя эти стрелки, можно занять пользователя, пока сохраняется файл большого размера, выводя на экран песочные часы, а потом их опять заменив обычной стрелкой.

Картинки. Загрузка картинок не сложнее, чем работа с курсорами и иконками. Для начала, Вы уже знакомы с процессом добавления файлов в проект и работы с ресурсами. Создавать новый проект не нужно. Достаточно скопировать файл с расширением .BMP в папку с вашим проектом и добавить его в проект через пункт меню Resource Add -> Resource.

В стандартном окне выбора ресурса выберите Bitmap. Обратите внимание, что компилятор рассматривает bitmap как картинку, аналогичную иконке. Если нажать New, он предложит нарисовать изображение, от чего лучше отказаться и выбрать Import. Указав файл рисунка, вы добавите его в проект. По умолчанию ему будет присвоено имя IDB_BITMAP1, которое можно изменить, например, на BMImage.

Файл ресурсов у вас теперь будет такой:

```
//First1.rc
#include "windows.h"
IDI_MYICON ICON "myicon.ico"
IDC_MYCURSOR "mycursor.cur"
BMImage BITMAP "1.bmp"
```

В проекте уже присутствует файл 1.bmp, который Вы назвали BMImage, теперь выведите на экран картинку.

```
LRESULT CALLBACK WndProc(...); //прототип WndProc

char szProgName[]="Имя программы"; //эта строка уже была
char szIconName[]="IDI_MYICON"; //и эта уже была
char szCursorName[]="IDC_MYCURSOR" //и эта была
char szImageName[]="BMImage"; //а эту строку добавьте

int WINAPI WinMain(..)
{
    HWND hWnd;
    MSG lpMsg;
    WNDCLASS w;

    hBitmap=LoadBitmap(hInstance, szImageName); //Получаем идентификатор
    картинки

    w.lpszClassName=szPrognamе;
    .....

    LRESULT CALLBACK WndProc(...)
    {
        HDC hdc, hmdc; //объявляем ещё один контекст изображения hmdc
        PAINTSTRUCT ps;
        BITMAP bm; //объявляем объект структуры BITMAP

        switch(messg) {

        case WM_PAINT:
            hdc=BeginPaint(hWnd, &ps);

            //здесь мы вставляем рисование картинки
            hmdc=CreateCompatibleDC(hdc); //Создаёт совместимый с оконным контекст
            памяти
            SelectObject(hmdc, hBitmap); //Выбирает объект картинку
            GetObject(hBitmap, sizeof(bm), (LPSTR) &bm); //получаем высоту и ширину
            картинки
            BitBlt(hdc, 10,10, bm.Width, bm.bmHeight, hmdc, 0,0, SRCCOPY); //Помещает
            картинку на экран в точку 10, 10
            //из памяти (hmdc)

            DeleteDC(hmdc);
            //Конец рисования картинки
```

Функция BitBlt широко применяется во всевозможных графических редакторах и программах. Её достоинство в том, что она копирует область из памяти куда угодно и с какими угодно размерами.

Рисовать Вы будете с помощью того же сообщения WM_PAINT. Все функции GDI имеют первым аргументом имя контекста HDC.

*Вывод текста. Здесь вы научитесь менять шрифт на пользовательский.

Функция **TextOut** выводит текстовую строку с заданными координатами.

```
BOOL TextOut(HDC hdc, int x, int y, LPCTSTR lpString, int cbString);
```

Параметры этой функции:

1. hdc - контекст графического устройства.
2. x, y - координаты, отсчёт которых ведётся из левого верхнего угла вниз и вправо.
3. lpString - сама строка
4. cbString - длина строки.

Функция SetTextAlign задаёт выравнивание текста.

```
UINT SetTextAlign(HDC hdc, UINT fMode);
```

Параметры этой функции:

1. hdc - контекст устройства
2. fMode - флаг выравнивания. Он отличается по вертикали (TA_TOP, TA_BASELINE, TA_BOTTOM) и по горизонтали. (TA_LEFT, TA_RIGHT, TA_CENTER).

Вот пример:

```
SetTextAlign(hdc, TA_CENTER|TA_BASELINE|TA_UPDATECP);
```

TA_UPADTECP - вывод текста с текущей позиции, игнорируя параметры в скобках.

*Смена шрифта.

Один из способов смены шрифта в API - заполнение структуры LOGFONT. Вот лишь некоторые из полей этой огромной структуры, которые нам понадобятся:

IfCharSet - задаёт набор требуемых символов (кодировку). Может принимать одно из значений: OEM_CHARSET, ANSI_CHARSET...

IfPitchAndFamily - задаёт ширину символов и определяет семейство шрифтов. Принимает значения: DEFAULT_PITCH - не имеет значения, имеет шрифт переменную ширину или фиксированную, FIXED_PITCH - ширина символов строго фиксирована, VARIABLE_PITCH - переменная ширина.

IfHeight - высота шрифта

IfWidth - ширина символов

IfItalic - если это поле не равно 0, шрифт будет курсивный

IfUnderline - если это поле не равно 0, шрифт будет подчёркнутый

IfFaceName - строка с именем шрифта.

Пусть в структуре LOGFONT заполнены нужные поля. Передайте указатель на неё, как аргумент функции CreateFontIndirect, которая возвращает дескриптор созданного шрифта hfont. Вот как это выглядит:

```
static LOGFONT lf; //создаём экземпляр структуры
lf.lfCharSet=DEFAULT_CHARSET; //заполняем некоторые поля
...
HFONT hFont=CreateFontIndirect(&lf); //создаём объект HFONT на основе
этой структуры
SelectObject(hdc, hFont); //Шрифт будет иметь силу только когда мы его
выберем
TextOut(hdc, 10,10, str, strlen(str)); //выводим строку. Она будет уже с новым
контекстом, а значит и шрифтом
```

Фрагмент программы, которая выводит строку с заданным шрифтом:

```
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM
wParam, LPARAM lParam)
{
char *str="Привет, Win API!!!";

{
PAINTSTRUCT ps; //экземпляр структуры рисования
```

```

HDC  hdc=BeginPaint(hWnd, &ps); //создаём контекст устройства и
показываем, что собираемся
//начать рисование

static LOGFONT lf; //создаём экземпляр LOGFONT
lf.lfCharSet=DEFAULT_CHARSET; //значение по умолчанию
lf.lfPitchAndFamily=DEFAULT_PITCH; //значения по умолчанию
strcpy(lf.lfFaceName,"Times New Roman"); //копируем в строку название
шрифта
lf.lfHeight=20; //высота
lf.lfWidth=10; //ширина
lf.lfWeight=40; //толщина

HFONT hFont=CreateFontIndirect(&lf); //Создали шрифт
SelectObject(hdc, hFont); //Он будет иметь силу только когда мы его выберем
SetTextColor(hdc, RGB(255,0,255)); //зададим цвет текста
SetBkColor(hdc, RGB(0,0,255)); //зададим цвет фона
TextOut(hdc, 100,100, str, strlen(str)); //выведем текст

DeleteObject(hFont); //выгрузим из памяти объект шрифта
EndPaint(hWnd, &ps); //закончили рисование
break;
}

```

SetTextColor - задаёт цвет шрифта, SetBkColor - цвет фона. Аргумент цвета типа RGB.

Теперь переходим к графическим примитивам. Сделайте заготовку, в которой будете пробовать все графические функции. Например, WM_PAINT:

```

case WM_PAINT :
//начинаем рисовать
hdc=BeginPaint(hWnd, &ps);
//здесь Вы обычно вставляете свой текст:
...

//обновляем окно
ValidateRect(hWnd, NULL);
//заканчиваем рисовать
EndPaint(hWnd, &ps);
break;

```

В исходном тексте всей программы, приведённой выше, Вам надо будет только заменить сообщение WM_PAINT. И добавлять графические функции.

1. Вывод точки.

SetPixel устанавливает заданный цвет в точке с указанными координатами:

```
COLORREF SetPixel(HDC hdc, int x, int y, COLORREF crColor);  
SetPixel(hdc, 10,10, RGB(0,0,0));
```

Функция GetPixel возвращает цвет в заданных координатах.

```
COLORREF Getpixel(hdc, int x, int y);
```

2. Рисование линий.

BOOL LineTo(hdc, int x, int y); //рисует линию от текущей позиции до места, указанного в аргументах. Чтобы изменить тип линии - меняется тип пера.

```
BOOL Line(hdc, int x1, int y1, int x2, int y2); //рисует линию от координат x1,  
y1 до x2, y2. Её надо задать самому:  
BOOL Line(hdc, int x1, int y1, int x2, int y2)  
{  
MoveToEx(hdc, x1, y1, NULL); //сделать текущими координаты x1, y1  
return LineTo(hdc, x2, y2);  
}
```

3. Дуга

```
BOOL Arc(hdc, int left, int top, int right, int bottom, int x1, int y1, int x2, int y2);
```

Первые четыре аргумента левый верхний и правый нижний углы прямоугольника, в который вписан эллипс. Остальные значения - координаты точек, от которых будут проведены прямые к центру эллипса. В местах пересечения первой и второй прямой с радиусом эллипса, начинается и кончается дуга.

4. Прямоугольник. По умолчанию прозрачный, тип его заливки определяется текущей кистью. По умолчанию она прозрачная.

```
BOOL Rectangle(hdc, int left, int top, int right, int bottom); //аргументы - это  
координаты левого верхнего и правого нижнего углов
```

5. Закруглённый прямоугольник. Его можно использовать, как импровизированную кнопку.

```
BOOL RoundRect(hdc, int left, int top, int right, int bottom, int width, int height);
```

Первые пять параметров совпадают с параметрами предыдущей функции. Далее width и height задают ширину и высоту эллипса, дуги которого ограничивают прямоугольник.

6. Кисти. Фигуры, которые будут рассмотрены дальше, выглядят лучше закрашенными. Рассмотрим, как задать свой стиль кисти. Кисть закрашивает какую-то область в какой-то цвет каким-то стилем.

Есть два способа объявить кисть. Первый - сплошную заливку, второй - указать стиль. Для этого существуют функции: CreateSolidBrush и CreateHatchBrush.

```
HBRUSH hBrush; //создаём объект-кисть
CreateSolidBrush(RGB(r,g,b)); //задаём сплошную кисть, закрашенную
цветом RGB
SelectObject(hdc, hBrush); //делаем кисть активной
```

А вот как объявить не сплошную кисть:

```
HBRUSH hBrush1;
CreateHatchBrush(int fnStyle, RGB(r,g,b));
SelectObject(hdc, hBrush1); //делаем кисть активной
```

Аргумент fnStyle принимает ряд константных значений:

HS_DIAGONAL - штрихует по диагонали
HS_CROSS - клеточка
HS_DIAGCROSS - диагональная сетка
HS_FDIAGONAL - по диагонали в другую сторону
HS_HORIZONTAL - горизонтальная "тельняшка"
HS_VERTICAL - вертикальная "забор"

Пример сообщения WM_PAINT с использованием кистей.

```
//сообщение рисования
case WM_PAINT :
//начинаем рисовать
hdc=BeginPaint(hWnd, &ps);
HBRUSH hBrush;
hBrush=CreateHatchBrush(HS_FDIAGONAL, RGB(255,0,0));
SelectObject(hdc,hBrush);

Ellipse(hdc, 100,100,200,300); //эллипс будет заштрихован

//обновляем окно
```

```
ValidateRect(hWnd, NULL);
```

```
//заканчиваем рисовать  
EndPaint(hWnd, &ps);  
break;
```

7. Перья. Задают стиль линий. Линия может быть жирной и тонкой, прерывистой и штрихпунктирной.

```
HPEN hPen;  
CreatePen(fnPenStyle, int width, RGB(r,g,b)); //аргументы: стиль, толщина,  
цвет  
SelectObject(hdc, hPen);
```

fnStyle может принимать следующие значения:

PS_SOLID - сплошная
PS_DASH - состоящая из точек
PS_DOT - состоящая из тире
PS_DASHDOT - "точка-тире"
PS_DASHDOTDOT - "тире-точка-точка-тире"
PS_NULL - невидимая
PS_INSIDEFRAME - обводка замкнутых фигур

Пример:

```
//сообщение рисования  
case WM_PAINT :  
//начинаем рисовать  
hdc=BeginPaint(hWnd, &ps);  
  
HPEN hPen1, hPen2, hPen3; //объявляем сразу три объекта-пера  
hPen1=CreatePen(PS_DASHDOT, 1, RGB(0,0,255)); //создаём всё три  
hPen2=CreatePen(PS_DASH, 1, RGB(255,0,255));  
hPen3=CreatePen(PS_DOT, 1, RGB(0,128,256));  
  
SelectObject(hdc, hPen1); //но в одном момент времени может быть только 1  
Rectangle(hdc, 10,10,100,100); //рисует фигуру соответствующим пером  
  
SelectObject(hdc, hPen2); //меняем перо  
Ellipse(hdc, 100,100,200,300); //рисует другим пером  
  
SelectObject(hdc, hPen3);  
LineTo(hdc, 200,100);
```

```
//обновляем окно  
  
ValidateRect(hWnd, NULL);  
//заканчиваем рисовать  
EndPaint(hWnd, &ps);  
break;
```

Важно понять, что можно создавать хоть 10 перьев с помощью CreatePen, но применить можно только 1 из них, для этого и нужен SelectObject.

6. Закрашенный прямоугольник

```
int FillRect(HDC hdc, CONST RECT *lprc, HBRUSH hbr);
```

lprc - закрашиваемый прямоугольник типа RECT.
hbr - кисть

Вот пример-фрагмент WM_PAINT:

```
RECT r; //экземпляр структуры RECT - координаты прямоугольника.  
r.left=100; //левый верхний угол  
r.top=100;  
r.right=200; //правый нижний  
r.right=300;  
FillRect(hdc, &r,(HBRUSH)(RGB(0,0,0)));
```

7. Прямоугольная рамка - как видите, немало функций для работы с прямоугольниками:

```
int FrameRect(HDC hdc, CONST RECT *lprc, HBRUSH hbr);
```

Применение аналогично предыдущей.

8. Инверсия значения цветов точек в заданной области

```
BOOL InvertRect(HDC hdc, CONST RECT *lprc);
```

9. Эллипс

```
BOOL Ellipse(HDC hdc, int x1, int y1, int x2, int y2);
```

координаты - это прямоугольник, в который вписывается эллипс

10. Хорда (сегмент эллипса) - параметры аналогичны Arc

```
BOOL Chord(HDC hdc, int left, int top, int right, int bottom, int x1, int y1, int x2, int y2);
```

Функция соединяет хордой точки начала и конца дуги эллипса и закрашивает выделенный сегмент текущей кистью.

11. Сектор эллипса.

```
BOOL Pie(HDC hdc, int left, int top, int right, int bottom, int x1, int y1, int x2, int y2);
```

12. Многоугольник. Есть много функций рисования многоугольников. Рассмотрим две. Рисования от вершины к вершине и рисования отрезками:

PolyDraw оперирует вершинами:

```
POINT poly[8];
BYTE polytype[8];
poly[0].x=375;
poly[0].y=375;

....

poly[7].x=400;
poly[7].y =400;

polytype[0]=PT_LINETO;

...

polytype[7]=PT_LINETO;

PolyDraw(hdc, poly, polytype, 8)
```

Функция Polyline рисует набором отрезков:

```
POINT poly[4];

poly[0].x =10;
poly[0].y =30;

poly[1].x =30;
poly[1].y =30;

poly[2].x =30;
```

```
poly[2].y =60;
```

```
poly[3].x=10;
```

```
poly[3].y=30;
```

```
Polyline(hdc, poly , 4);
```

Теперь рассмотрим, как мышь взаимодействует с приложением Win API, обработав простое сообщение, связанное с нажатием кнопки. В данном примере будет обрабатываться нажатие левой кнопки мыши, но в любой момент можно изменить обработку на правую кнопку, двойной щелчок и другие действия.

Необходимо также отследить координаты, где произошло нажатие. При нажатии левой или правой кнопки мыши на экране появится сообщение: "Поздравляю, вы уже умеете работать с мышью...", при этом текст будет отображаться в той точке, где произошло нажатие.

Для реализации данной функции потребуется обработать соответствующее сообщение мыши и использовать функции для вывода текста в указанной позиции.

```
#include "windows.h"

//Создаём прототип функции окна
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);

char szIconName[]="MYICON"; //имя иконки
char szProgName[]="Progame"; //имя программы
char szText[]="Поздравляю, теперь вы умеете работать с мышью!";

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPSTR lpCmdLine, int nCmdShow)
{
    HWND hWnd; //идентификатор окна
    MSG lpMsg; //идентификатор сообщения
    WNDCLASS w; //создаём экземпляр структуры WNDCLASS

    //И начинаем её заполнять
    w.lpszClassName=szProgName; //имя программы - объявлено выше
    w.hInstance=hInstance; //идентификатор текущего приложения
    w.lpfnWndProc=WndProc; //указатель на функцию окна
    w.hCursor=LoadCursor(NULL, IDC_ARROW); //загружаем курсор
    w.hIcon=LoadIcon(hInstance, szIconName); //загружаем нашу иконку
    w.lpszMenuName=0; //меню пока не будет
    w.hbrBackground=(HBRUSH)GetStockObject(WHITE_BRUSH); //цвет фона
    окна
    w.style=CS_HREDRAW|CS_VREDRAW; //стиль - перерисовываемое по x и
    по y
    w.cbClsExtra=0;
    w.cbWndExtra=0;
```

```

//Если не удалось зарегистрировать класс окна - выходим
if(!RegisterClass(&w))
return 0;

//Создадим окно в памяти, заполнив аргументы CreateWindow
hWnd=CreateWindow(szProgName, //Имя программы
"Моя первая программа!", //Заголовок окна
WS_OVERLAPPEDWINDOW, //Стиль окна - перекрывающееся
100, //положение окна на экране по x
100, //по y
500, //размеры по x
400, //по y
(HWND)NULL, //идентификатор родительского окна
(HMENU)NULL, //идентификатор меню
(HINSTANCE)hInstance, //идентификатор экземпляра программы
(HINSTANCE)NULL); //отсутствие дополнительных параметров

//Выводим окно из памяти на экран
ShowWindow(hWnd, nCmdShow);
//Обновим содержимое окна
UpdateWindow(hWnd);

//Цикл обработки сообщений

while(GetMessage(&lpMsg, NULL, 0, 0)) { //Получаем сообщение из очереди
TranslateMessage(&lpMsg); //Преобразует сообщения клавиш в символы
DispatchMessage(&lpMsg); //Передаёт сообщение соответствующей функции
окна
}
return(lpMsg.wParam);
}

//Функция окна
LRESULT CALLBACK WndProc(HWND hWnd, UINT messg,
WPARAM wParam, LPARAM lParam)
{
HDC hdc; //создаём контекст устройства
PAINTSTRUCT ps; //создаём экземпляр структуры графического вывода

//Цикл обработки сообщений
switch(messg)
{

int x,y; //координаты мыши

```

```

case WM_RBUTTONDOWN:
case WM_LBUTTONDOWN:
char *str;
HDC hDC;

hDC=GetDC(hWnd);
x=LOWORD(lParam);
y=HIWORD(wParam);

TextOut(hDC, x, y, szText, strlen(szText));

break;

//сообщение выхода - разрушение окна
case WM_DESTROY:
PostQuitMessage(0); //Посылаем сообщение выхода с кодом 0 - нормальное
завершение
break;

default:
return(DefWindowProc(hWnd, messg, wParam, lParam)); //освобождаем
очередь приложения от нераспознанных
}
return 0;
}

```

Усложним задачу. Нарисуем кнопку и отследим нажатие кнопки на неё. При этом кнопка будет нарисована Вами. Процедура рисования кнопки будет происходить по сообщению WM_PAINT, а отслеживать нажатие будем WM_LBUTTONDOWN. Над кнопкой добавим картинку, чтобы интерфейс был дружелюбный.

```

//SimpleButton.rc
#include "windows.h"

MYICON ICON "myicon.ico"
MyImage IMAGE "1.bmp"

```

```

//SimpleButton.cpp
#include "windows.h"

//Создаём прототип функции окна

```

```

LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);

char szIconName[]="MYICON";
char szImageName[]="MyImage";
char szProgName[]="Progname";

HBITMAP hBitmap;

//объявляем имя программы
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPSTR lpCmdLine, int nCmdShow)
{

HWND hWnd; //идентификатор окна
MSG lpMsg;
hBitmap=LoadBitmap(hInstance, szImageName);

WNDCLASS w; //создаём экземпляр структуры WNDCLASS

//И начинаем её заполнять
w.lpszClassName=szProgName; //имя программы - объявлено выше
w.hInstance=hInstance; //идентификатор текущего приложения
w.lpfnWndProc=WndProc; //указатель на функцию окна
w.hCursor=LoadCursor(NULL, IDC_ARROW); //загружаем курсор
w.hIcon=LoadIcon(hInstance, szIconName);
w.lpszMenuName=0; //меню пока не будет
w.hbrBackground=(HBRUSH)GetStockObject(WHITE_BRUSH); //цвет фона
окна
w.style=CS_HREDRAW|CS_VREDRAW; //стиль - перерисовываемое по x и
по y
w.cbClsExtra=0;
w.cbWndExtra=0;

//Если не удалось зарегистрировать класс окна - выходим
if(!RegisterClass(&w))
return 0;

//Создадим окно в памяти, заполнив аргументы CreateWindow
hWnd=CreateWindow(szProgName, //Имя программы
"Моя первая программа!", //Заголовок окна
WS_OVERLAPPEDWINDOW, //Стиль окна - перекрывающееся
100, //положение окна на экране по x
100, //по y
500, //размеры по x
400, //по y

```

```

(HWND)NULL, //идентификатор родительского окна
(HMENU)NULL, //идентификатор меню
(HINSTANCE)hInstance, //идентификатор экземпляра программы
(HINSTANCE)NULL); //отсутствие дополнительных параметров

//Выводим окно из памяти на экран
ShowWindow(hWnd, nCmdShow);
//Обновим содержимое окна
UpdateWindow(hWnd);

//Цикл обработки сообщений

while(GetMessage(&lpMsg, NULL, 0, 0)) { //Получаем сообщение из очереди
TranslateMessage(&lpMsg); //Преобразует сообщения клавиш в символы
DispatchMessage(&lpMsg); //Передаёт сообщение соответствующей функции
окна
}
return(lpMsg.wParam);
}

//Функция окна
LRESULT CALLBACK WndProc(HWND hWnd, UINT messg, WPARAM
wParam, LPARAM lParam)
{

//Координаты кнопки
int x1=180;
int y1=250;
int x2=320;
int y2=300;

//Цикл обработки сообщений
switch(messg)
{

int x,y; //координаты мыши
HDC hdc, hmdc;
BITMAP bm;
RECT rect;
PAINTSTRUCT ps;
LOGFONT lf;
HFONT hFont;

case WM_PAINT:

```

```

hdc=BeginPaint(hWnd, &ps); //Начинаем рисовать

//Рисуем картинку:
hmdc=CreateCompatibleDC(hdc);
SelectObject(hmdc, hBitmap);
GetObject(hBitmap, sizeof(bm), (LPSTR)&bm);
BitBlt(hdc, 10,10, bm.bmWidth+100, bm.bmHeight, hmdc, 0,0,SRcinvert);
DeleteDC(hmdc);

HBRUSH hBrush; //Создаём кисть

//Рамка кнопки
hBrush=CreateSolidBrush(RGB(255,255,0));
SelectObject(hdc, hBrush);
RoundRect(hdc,x1,y1,x2,y2,20,20);

//Сама кнопка
hBrush=CreateSolidBrush(RGB(0,255,0));
SelectObject(hdc, hBrush);
RoundRect(hdc,185,255,315,295,20,20);

//Задаём параметры шрифта
lf.lfCharSet=DEFAULT_CHARSET; //значение по умолчанию
lf.lfPitchAndFamily=DEFAULT_PITCH; //значения по умолчанию
strcpy(lf.lfFaceName,"Times New Roman"); //копируем в строку название
шрифта
lf.lfHeight=20; //высота
lf.lfWidth=10; //ширина
lf.lfWeight=FW_BOLD; //толщина
lf.lfEscapement=0; //не повёрнутый
lf.lfStrikeOut=0; //на перечёркнутый
lf.lfUnderline=0; //не подчёркнутый

hFont=CreateFontIndirect(&lf); //Создали шрифт
SelectObject(hdc, hFont); //Он будет иметь силу только когда мы его выберем

SetTextColor(hdc, RGB(255,0,0)); //зададим цвет текста

SetBkMode(hdc, TRANSPARENT); //текст будет без фона
TextOut(hdc, 218, 263,"Enter", 5); //текст кнопки

EndPaint(hWnd, &ps);
break;

//сообщение мыши

```

```

case WM_LBUTTONDOWN:

HDC hDC; //получаем контекст

hDC=GetDC(hWnd);

x=LOWORD(lParam);//координаты с которыми нажата мышь - аргументы
WndProc
y=HIWORD(lParam);

//Если нажатие вписалось в координаты кнопки
if((x>x1)&&(x<x2)&&(y>y1)&&(y<y2))
{
MessageBox(hWnd, "Поздравляю, с мышью вы разобрались!", "Yes-s-s!",
MB_OK|MB_ICONINFORMATION);
}

break;

//сообщение выхода - разрушение окна
case WM_DESTROY:
DeleteObject(hBrush); //уничтожаем кисть
DeleteObject(hFont); //уничтожаем шрифт

PostQuitMessage(0); //Посылаем сообщение выхода с кодом 0 - нормальное
завершение

break;

default:
return(DefWindowProc(hWnd, messg, wParam, lParam)); //освобождаем
очередь приложения от нераспознанных
}
return 0;
}

```

Для отслеживания координат мыши при нажатии используется параметр lParam, который состоит из двух частей: HIWORD и LOWORD. В данном контексте, если поступает сообщение WM_LBUTTONDOWN, координаты точки окна, где произошло нажатие, можно получить следующим образом: (LOWORD(lParam)) — для координаты x, и (HIWORD(lParam)) — для координаты y.

Важно помнить, что после завершения работы с объектами, такими как шрифты, кисти и пера, необходимо их уничтожить. Если этого не сделать, они

останутся в памяти и могут оказывать влияние на другие программы. Для корректного освобождения ресурсов в обработчике сообщения WM_DESTROY следует использовать функцию DeleteObject для удаления шрифта и кисти.

```
DeleteObject(hBrush); //уничтожаем кисть  
  
DeleteObject(hFont); //уничтожаем шрифт
```

Теперь создайте программу, которая будет выводить на экран все символы на клавиатуре, которые мы нажали. Используем сообщение от WM_CHAR.

Создайте новый проект MyChar. Создайте файл MyChar.cpp и скопируйте в него исходник-шаблон API. Добавьте в него библиотечный файл: **#include <stdio.h>**. Удалите всё сообщение WM_PAINT. Теперь вместо него добавьте несколько строчек:

```
...WndProc(...)  
{  
char *szChar=""; //объявим строковую переменную  
  
...  
//Сообщение символьного нажатия  
case WM_CHAR:  
  
hdc1=GetDC(hWnd); //получим контекст изображения  
TextOut(hdc1, 1, 1, " ", 2); //сотрём предыдущий символ  
sprintf(szChar, "%c", (char)wParam); //скопируем код нажатой клавиши в строку  
TextOut(hdc1, 1,1, szChar, strlen(szChar)); //выведем строку на экран  
  
ReleaseDC(hWnd, hdc1);  
break;
```

Так программа обрабатывает символьные клавиши.

WM_KEYDOWN позволяет Вам обрабатывать нажатие совершенно любой клавиши! Сообщение WM_KEYDOWN работает с кодами клавиш в 16-ричной системе. Например, Enter будет 0xd, а 0x20 - пробел.

Префикс каждой виртуальной клавиши: VK_, а дальше: VK_RETURN - Enter, VK_TAB - Tab и т.д. Вы можете написать простую программу, вроде той, что Вы уже писали, только добавить в неё отображение 16-ричного кода клавиши. Это просто.

```

#include<stdio.h>
#include<conio.h>

void main()
{
int key;
do {

key=getch();
clrscr();
textcolor(15);
printf("Code = %d Symbol =%c Hex=0x%x\n", key,key,key);

}while(key!=27);

}

```

С помощью этой программы Вы всегда будете знать, какой клавише соответствует какой код. Ниже приведен список некоторых кодов виртуальных клавиш:

Символическое имя	Код клавиши	Клавиша на клавиатуре.
VK_CANCEL		<CTRL>+<BREAK>
VK_BACK		BACKSPACE
VK_TAB		TAB
VK_CLEAR		5 доп. клав.
VK_RETURN		ENTER
VK_SHIFT		SHIFT
VK_CONTROL		CTRL
VK_MENU		ALT
VK_PAUSE		PAUSE
VK_CAPITAL		CAPS LOCK
VK_ESCAPE		ESC
VK_SPACE	0x20	ПРОБЕЛ
VK_PRIOR	0x21	PAGE UP
VK_NEXT	0x22	PAGE DOWN
VK_END	0x23	END
VK_HOME	0x24	HOME
VK_LEFT	0x25	LEFT
VK_UP	0x26	UP
VK_RIGHT	0x27	RIGHT
VK_DOWN	0x28	DOWN
VK_SNAPSHOT	0x2a	PRINT SCREEN

VK_INSERT	0x2d	INSERT
VK_DELETE	0x2e	DELETE
	0x30	A
	0x31	B
	0x5a	Z
VK_NUMPAD0	0x60	<0> на доп. клав.
VK_NUMPAD1	0x61	<1> на доп. клав.
VK_NUMPAD9	0x69	<9>
VK_ADD	0x6b	<+>
VK_MULTIPLY	0x6a	<*>
VK_SUBTRACT	0x6d	<->
VK_F1	0x70	F1
VK_F2	0x71	F2
VK_F12	0x7b	F12
VK_NUMLOCK	0x90	NUM LOCK
VK_SCROLL	0x91	SCROLL LOCK

Добавьте меню. Меню можно добавить двумя способами: вписать в файл ресурсов самим или добавить, как новый ресурс, через меню Add -> Resource.

Создайте новый проект, который назовите MyMenu. В пустой проект добавьте файл с шаблоном нашей первой программы, который назовите MyMenu.cpp. Создайте пустой файл ресурсов. Сейчас в него Вы запишете код простого меню.

```
#include "windows.h"

#define IDM_EXIT 1 //каждому пункту меню соответствует идентификатор,
имя которого начинается с IDM
// и номер, который определяете вы сами. Номера не должны повторяться!

MainMenu MENU //имя нашего меню MainMenu

POPUP "Файл" //Всплывающее меню со словами: "Файл"
{

    MENUITEM "Выход", IDM_EXIT //Пункт меню выход. Ему соответствует
идентификатор IDM_EXIT

}
```

Когда Вы сохраните файл RC, то увидите, что во вкладке Resource вашего проекта, появится меню с именем MAINMENU. Теперь объявите имя этого меню, как Вы это делали для иконок, курсоров и картинок:

```
char szMenu[]="MainMenu"; //разместите эту строку в начале программы, до WinMain

w.lpszmenuName=szMenu; //теперь поменяем ещё одну строчку. Наконец-то в структуре WNDCLASS появится //ссылка на настоящее меню!
```

После компиляции и запуска программы будет видно, что окно получило небольшое, но привлекательное меню с одним пунктом: "Файл", содержащим вкладку "Выход". На данный момент этот пункт не выполняет никаких действий. Поскольку меню является дочерним окном, для него необходим собственный цикл обработки сообщений, который будет реагировать на выбор соответствующих пунктов. Например, при выборе "Выход" программа должна завершить свою работу, а при выборе "О программе..." следует отображать небольшое диалоговое окно. Все эти действия прописываются в цикле обработки сообщений.

Далее необходимо дополнить файл ресурсов, что приведёт к увеличению количества пунктов в меню. Конечной целью является создание утилиты, которая позволит узнать информацию о свободном месте на диске, включая название текущей папки. В одном из пунктов меню будет доступна информация о текущей дате и времени, а в другом — имя пользователя и имя компьютера. Эти данные окажутся весьма полезными в дальнейшем.

```
#include "windows.h"

//Каждому пункту по личному номеру!
#define IDM_EXIT 1
#define IDM_DSPACE 2
#define IDM_FREE 3
#define IDM_CURRENT 4
#define IDM_ABOUT 5

MainMenu MENU
{
    //1-й пункт меню
    POPUP "&Файл"
    {
        MENUITEM "В&ыход", IDM_EXIT
    }
}
```

```
//2-й пункт меню
POPUP "&Диск"
{
MENUITEM "Всего на диске", IDM_DSPACE
MENUITEM "Свободное место", IDM_FREE
MENUITEM "Текущая папка", IDM_CURRENT
}

//3-й пункт меню
POPUP "&Помощь"
{
MENUITEM "О программе", IDM_ABOUT
}

}
```

Каждый пункт (POPUP) имеет вкладки MENUITEM, которые содержат сам текст меню и идентификатор. Каждый идентификатор начинается с IDM_ (Identifier menu). В начале текста всем идентификаторам присвоены номера.

Если в меню нужно вставить разделитель, которым обычно отделяют пункт "Выход" от остальных в меню "Файл", пишут MENUITEM SEPARATOR, без идентификатора. Если начиная со следующего пункта, нужно пустить пункты меню во второй столбик, пишут MENUBARBREAK:

```
MENUITEM "&Disk space", IDM_FREE, MENUBARBREAK
```

Если надо сделать ещё одно подменю, применима следующая конструкция:

```
POPUP "&User Name"
{
MENUITEM "User number", IDM_NUM

POPUP "User status"
{
MENUITEM "Administrator", IDM_ADMIN
MENUITEM "Guest", IDM_GUEST
}
}
```

Таким образом, вкладка "User status" будет содержать подменю из двух пунктов: Administrator и Guest.

И ещё. Вы, наверное, обратили внимание на знак &, который Вы можете увидеть между слов. Смысл его такой: чтобы дать доступ пользователю к пунктам меню с клавиатуры, пишем допустим &Файл. Тогда в меню слово "Файл", будет записано, как Файл, и если Вы нажмете Alt+Ф, то перейдете на этот пункт. Признаком хорошего стиля служит введение во все пункты быстрого доступа.

Теперь вы знаете всё, что нужно о синтаксисе меню в файле ресурсов.

Вы, наверное, уже задумывались над тем, какое сообщение обрабатывает выбор пунктов меню. Сейчас Вы познакомитесь с новым для нас сообщением **WM_COMMAND**. Оно вызывается тогда, когда пользователь нажимает одну из строчек меню.

```
switch(messg)
{
    case WM_COMMAND:

        switch(LOWORD(wParam)) {

            case IDM_ADMIN: { //Выбор пункта с идентификатором IDM_ADMIN
                .... //Реакция программы на нажатие
                break;
            }

            case IDM_GUEST: { //Выбор другого пункта
                ....
                break;
            }
        }

        case WM_PAINT:

            ....
        }
}
```

Перехватываем значение wParam при поступлении WM_COMMAND. Разделяем по пунктам меню и выполняем соответствующие процедуры.

Далее приведен полный текст программы с использованием меню:

```
#include <windows.h>
#include <stdio.h>
```

```

#define IDM_EXIT 1
#define IDM_DSPACE 2
#define IDM_FREE 3
#define IDM_CURRENT 4
#define IDM_ABOUT 5
#define IDM_DATE 6
#define IDM_TIME 7
#define IDM_ACOMP 8

LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);

char szProgName[]="Имя программы";
char str[]="Мой текст!";
char szMenu[]="MainMenu";
char szMessage[]="Выберите нужный пункт меню.";
short sLen;

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPSTR lpszCmdLine, int nCmdShow)
{
    HWND hWnd;
    MSG lpMsg;

    WNDCLASS w; //создаём экземпляр структуры WNDCLASS
    //И начинаем её заполнять
    w.lpszClassName=szProgName; //имя программы - объявлено выше
    w.hInstance=hInstance; //идентификатор текущего приложения
    w.lpfnWndProc=WndProc; //указатель на функцию окна
    w.hCursor=LoadCursor(NULL, IDC_ARROW); //загружаем курсор
    w.hIcon=LoadIcon(NULL, IDI_APPLICATION); //иконки у нас не будет пока
    w.lpszMenuName=szMenu; //и меню пока не будет
    w.hbrBackground=(HBRUSH)GetStockObject(WHITE_BRUSH); //цвет фона
    окна
    w.style=CS_HREDRAW|CS_VREDRAW; //стиль - перерисовываемое по x и
    по y
    w.cbClsExtra=0;
    w.cbWndExtra=0;

    //Если не удалось зарегистрировать класс окна - выходим
    if(!RegisterClass(&w))
        return 0;

    //Создадим окно в памяти, заполнив аргументы CreateWindow

```

```

hWnd=CreateWindow(szProgName,
"Демонстрация меню в Win API!",
WS_OVERLAPPEDWINDOW,
100,
100,
500,
400,
(HWND)NULL,
(HMENU)NULL,
hInstance,
(LPSTR)NULL);

//Выводим окно из памяти на экран
ShowWindow(hWnd, nCmdShow);
//Обновим содержимое окна
UpdateWindow(hWnd);

while(GetMessage(&lpMsg, NULL, 0, 0)) {
TranslateMessage(&lpMsg);
DispatchMessage(&lpMsg);
}
return(lpMsg.wParam);
}

//Функция окна
LRESULT CALLBACK WndProc(HWND hWnd, UINT messg,
WPARAM wParam, LPARAM lParam)
{
HDC hdc; //создаём контекст устройства
PAINTSTRUCT ps; //создаём экземпляр структуры графического вывода

//дискровая информация
unsigned long lSectInClust, lByteInSect, lNumFreeClust, lNumClust, lTotal, lBuf,
lBuf1;
char szCurrent[MAX_PATH];

//информация о дате и времени
SYSTEMTIME st;
WORD month, day, year, hour, min, sec;

//идентификация имени пользователя и компьютера
char ComputerName[MAX_COMPUTERNAME_LENGTH + 1];
unsigned long len_ComputerName = MAX_COMPUTERNAME_LENGTH + 1;
char UserName[256 + 1];
unsigned long len_UserName = 256 + 1;

```

```
BOOL comp;

//Цикл обработки сообщений
switch(messg)
{

case WM_COMMAND:
switch(LOWORD(wParam)){

//Выход из программы
case IDM_EXIT: DestroyWindow(hWnd);

//Сколько места на диске?
case IDM_DSPACE:
GetDiskFreeSpace(NULL, &lSectInClust, &lByteInSect, &lNumFreeClust,
&lTotal);
lBuf=lSectInClust*lByteInSect*lTotal;
sLen=sprintf(szMessage, "Всего на диске: %ld Мб", lBuf/1000000);
break;

//Свободно на диске
case IDM_FREE:
GetDiskFreeSpace(NULL, &lSectInClust, &lByteInSect, &lNumFreeClust,
&lTotal);
lBuf=lSectInClust;
lBuf*=lByteInSect;
lBuf*=lNumFreeClust;
sLen=sprintf(szMessage, "На диске свободно: %ld Мб", lBuf/1000000);
break;

//Текущая папка
case IDM_CURRENT:
GetCurrentDirectory(MAX_PATH, szCurrent);
sLen=sprintf(szMessage, "Тек. папка %s", szCurrent);
break;

//Текущее время
case IDM_TIME:
GetSystemTime(&st);

min=st.wMinute;
sec=st.wSecond;
hour=st.wHour;
if(min<10)
```

```
sLen=sprintf(szMessage, "Время: %d:0%d:0%d", hour, min, sec);
else
sLen=sprintf(szMessage, "Время: %ld:%d:%d", hour, min, sec);
break;

//Текущая дата
case IDM_DATE:
GetSystemTime(&st);
year=st.wYear;
month=st.wMonth;
day=st.wDay;
if(month<10)
sLen=sprintf(szMessage, "Дата: %d/0%d/0%d", day, month, year);
else
sLen=sprintf(szMessage, "Дата: %d/%d/%d", day, month, year);
break;

//Имя компьютера
case IDM_ACOMP:

comp = GetComputerName(ComputerName, &len_ComputerName);

if( comp != 0 )
{
sLen=sprintf(szMessage, "%s", ComputerName);
}
else
MessageBox(hWnd, "Имя компьютера не доступно!", "Сообщение:",
MB_OK|MB_ICONSTOP);

break;

//Имя пользователя
case IDM_ABOUT:

comp = GetUserNameA (UserName, &len_UserName);

if( comp != 0 )
{
sLen=sprintf(szMessage, "%s", UserName);
}
else
MessageBox(hWnd, "Имя пользователя не доступно!", "Сообщение...",
MB_OK|MB_ICONSTOP);
```

```

break;

}
InvalidateRect(hWnd, NULL, TRUE);
break;

//Вывод строки на экран
case WM_PAINT:
hdc=BeginPaint(hWnd, &ps);
TextOut(hdc, 10,10, szMessage, sLen);
ValidateRect(hWnd, NULL);
EndPaint(hWnd, &ps);
break;

//Завершение работы окна
case WM_DESTROY:
PostQuitMessage(0);
break;

default:
return(DefWindowProc(hWnd, messg, wParam, lParam));
}
return 0;
}

```

Смысл программы такой. Строка текста выводится на экран только при поступлении сообщения WM_PAINT. При выборе каждого пункта, заполняется строка (szMessage) какой-либо информацией и вычисляется её длина (sLen). В конце цикла обработки сообщения вызывается функция InvalidateRect:

```
InvalidateRect(hWnd, NULL, TRUE);
```

Тем самым мы передаём окну сообщение WM_PAINT. Окно обновляется и снова выводится строка с сообщением, только содержание её поменялось. В него теперь записана совсем другая информация.

Этот приём неоднократно будет помогать. Диалоговые окна, в которые Вы будете вводить числа, тоже будут вызывать перерисовку, и от этого будет меняться график на экране, хотя сообщение WM_PAINT, в котором все обычно рисуют, остаётся нетронутым.

Теперь о служебных функциях, которые мы использовали в нашей программе.

```
1. Дискковые операции.  
case IDM_DSPACE:  
GetDiskFreeSpace(NULL, &lSectInClust, &lByteInSect, &lNumFreeClust,  
&lTotal);  
lBuf=lSectInClust*lByteInSect*lTotal;  
sLen=sprintf(szMessage, "Всего на диске: %ld Мб", lBuf/1000000);  
break;
```

Выше были объявлены переменные: lSectInClust (секторов в кластере), lByteInSect (байт в секторе), lNumFreeClust (число свободных кластеров), lTotal (всего число кластеров). Функция GetDiskFreeSpace заполнила эти переменные реальными значениями этих параметров вашего жёсткого диска. lBuf - буфер, в который мы записали произведение: Секторов в кластере X число байт в кластере X общее число кластеров. Таким образом lBuf будет содержать сколько байт на жёстком диске. Если изменить в формуле общее число кластеров на общее число свободных кластеров, мы узнаем, сколько свободных байт на винчестере.

```
2. Текущая папка  
GetCurrentDirectory(MAX_PATH, szCurrent);  
sLen=sprintf(szMessage, "Тек. папка %s", szCurrent);
```

Как уже говорилось, MAX_PATH - константа, в которую записана длина максимально допустимого пути. Функция GetCurrentDirectory записывает в строку szCurrent текущий путь. Функция полностью поддерживает длинные имена.

```
3. Системное время  
case IDM_TIME:  
GetSystemTime(&st);  
  
min=st.wMinute;  
sec=st.wSecond;  
hour=st.wHour;  
if(min<10)  
sLen=sprintf(szMessage, "Время: %d:0%d:%d", hour, min, sec);  
else  
sLen=sprintf(szMessage, "Время: %ld:%d:%d", hour, min, sec);  
break;
```

Структура, отвечающая за время, прописана в библиотеке time.h. Она называется SYSTEMTIME. Создав экземпляр этой структуры, мы заполняем его функцией GetSystemTime. Затем присваиваем нашим переменным min, hour, sec значения отдельных полей структуры st.wMinute, st.wSecond, st.wHour. Если число минут меньше десяти, время будет отражаться так:

1:9:27, что неприемлемо. Поэтому неплохо будет отследить число минут меньшее 10 и добавлять ноль. Можно это сделать и для секунд, но они не так бросаются в глаза.

С датой всё то же самое, только используем другие поля: st.wYear, st.wMonth. Кстати есть ещё поля: st.wDayOfWeek (день недели), st.wMilliseconds - число миллисекунд во времени.

4. Сведения окружения. Окружением называются окружающие Вас важные данные: имя пользователя (в многопользовательских системах типа Windows NT это очень актуально), имя папки временных файлов Temp, имя компьютера, которое ему дали при установке Windows, путь в котором размещена папка Windows... Все эти данные используются программами, которые привносят в систему свои DLL и им просто необходимо знать путь системной папки. Тем не менее, получить доступ к этим данным предельно просто. Рассмотрим работу с такими функциями на примере получения имени компьютера.

```
char ComputerName[MAX_COMPUTERNAME_LENGTH + 1]; //строка имя
компьютера
unsigned long len_ComputerName = MAX_COMPUTERNAME_LENGTH + 1;
//длина имени
BOOL comp; //признак того, можно ли получить эту информацию
...
case IDM_ACOMP: //выбрали соответствующий пункт меню

comp = GetComputerName(ComputerName, &len_ComputerName);

if( comp != 0 ) //Если информация доступна,
{
sLen=sprintf(szMessage, "%s", ComputerName); //формируем строку
}
else
//сообщение об ошибке, если нет доступа к переменным окружения
MessageBox(hWnd, "Имя компьютера не доступно!", "Сообщение:",
MB_OK|MB_ICONSTOP);

break;
```

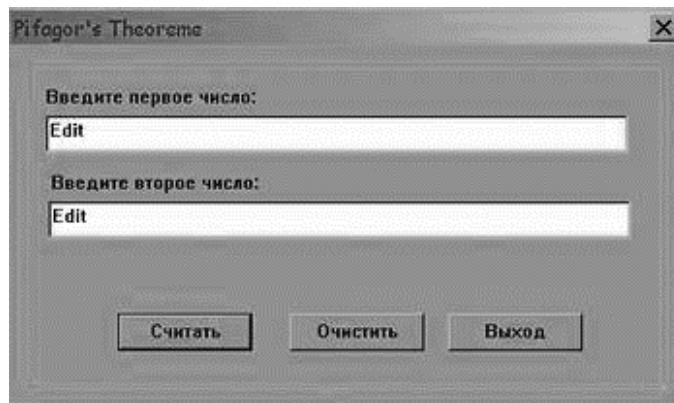
До сих пор Вы пользовались только готовыми диалоговыми окнами: сообщениями об ошибке, информационными сообщениями и др. Теперь создайте собственное всплывающее диалоговое окно, в которое пользователю будет вежливо предложено ввести данные. После того, как пользователь введёт в поля ввода два вещественных числа, программа посчитает теорему Пифагора и выдаст ответ на экран.

Создайте пустой проект Win API. Добавьте в него исходник-шаблон.

Диалоговое окно, с которым Вы будете работать, можно также включать через Add -> Resource -> Dialog -> New, а можно набирать текст вручную. Имейте в виду, что редактор ресурсов предназначен в первую очередь, всё-таки для MFC приложений, поэтому в него будет включено помимо нужной информации, масса определений, констант и директив препроцессора. Из-за этого он может некорректно работать с приложением Win API, вплоть до того, что вообще не вызываться. Эта тема слабых мест Microsoft довольно болезненна и редко обсуждается. Тем не менее, в любой книге по Win API авторы приводят тексты RC файлов непохожие на те, что генерирует редактор ресурсов. Всё же, можете включить в файл автоматически сгенерированный диалог.

На экране появится форма, в которую вы можете перетаскивать готовые элементы управления, расположенные справа:

Вот что должно получиться:



Если Вы работаете в редакторе ресурсов, перетащите в окно два элемента **Static text** - это будут заголовки над полями ввода. Поменяйте в них текст с помощью окна Properties. Потом перетащите два **Edit Box**. В окне Properties (правая кнопка мыши) измените их идентификаторы на IDC_VALUE1 и IDC_VALUE2 соответственно. Перетащите три кнопки и назовите их: IDC_COUNT, IDC_CLEAR, IDC_EXIT. Смените через Properties шрифт в окне на "System", высотой 10 и поменяйте заголовок (Caption) на Pifagor's Theoreme. Само окно будет называться IDD_PIFAGORUS. Всё! Окно готово.

Итак, файл ресурсов описывает и диалоговые окна. Все компоненты окна - кнопки, элементы управления и даже статический текст, имеют свои идентификаторы, координаты в окне и номера. Вот пример такого файла:

```
//Pifagor.rc
#include "windows.h"

#define IDD_PIFAGORUS 101
#define IDC_VALUE1 1000
#define IDC_VALUE2 1001
#define ID_EXIT 1002
#define ID_COUNT 1003
#define ID_CLEAR 1004
#define IDC_STATIC -1

#define IDM_ENTER 1005
#define IDM_EXIT 1006

IDD_PIFAGORUS DIALOG DISCARDABLE 0, 0, 250, 134 //Параметры
диалогового окна
STYLE DS_MODALFRAME | WS_POPUP | WS_CAPTION | WS_SYSMENU
//стили окна
CAPTION "Pifagor's Theoreme" //заголовок окна
FONT 10, "System" //шрифт
{
  DEFPUSHBUTTON "Считать",ID_COUNT,40,100,50,14 //Тип кнопки,
надпись на ней, идентификатор, её координаты
PUSHBUTTON "Очистить",ID_CLEAR,104,100,50,14
LTEXT "Введите первое число:",IDC_STATIC,13,15,80,8 //Статический
текст
EDITTEXT IDC_VALUE1,13,26,217,14,ES_AUTOHSCROLL //поле ввода
LTEXT "Введите второе число:",IDC_STATIC,15,47,78,8
EDITTEXT IDC_VALUE2,14,58,217,14,ES_AUTOHSCROLL
PUSHBUTTON "Выход",ID_EXIT,163,100,50,14
}

//И небольшое такое меню

PifagrorusMenu MENU
{
  POPUP "&Файл"
  {
    MENUITEM "&Ввод данных...", IDM_ENTER
    MENUITEM "Вы&ход", IDM_EXIT
```

```
}  
}
```

Директивы `#define` можно включать, как в файл RC, так и в исходный текст программы для того, чтобы не создавать третий файл, который обычно называют `resource.h`. Вы можете в нём указать все Ваши `#define` и включить на него ссылку как в файле RC, так и в файле CPP:

```
#include "resource.h"
```

Только когда Вам нужно будет срочно узнать, какие у вас есть пункты, придётся открывать этот файл. Итак, меню создано. Создано и диалоговое окно. Они появились во вкладке ресурсов. Теперь поговорим об особенностях работы с диалоговыми окнами.

Таймер. Это прозвучит странно, но Вы каждый день работаете с приложениями, использующими таймер. Более того, одна и та же программа может одновременно работать сразу с несколькими таймерами. Что делает таймер? Сразу после своей инициализации он начинает со строгой периодичностью (её вы задаёте сами) генерировать сообщение `WM_TIMER`. В этом сообщении, разумеется, может быть всё, что вам заблагорассудится. Вы можете рисовать в окне красный фонарь, который будет мигать "Alarm alarm!", можете задать бегущую строку.

Вызов - инициализацию таймера производит функция `SetTimer`, которая следует на начальном этапе, где-то за `ShowWindow`. Синтаксис у неё такой:

```
UINT SetTimer(HWND hWnd, UINT nID, UINT wLength, TIMEPROC  
lpTFunc);
```

`hWnd` - идентификатор окна

`nID` - порядковый номер запущенного таймера

`wLength` - длина интервала, через который будет вызываться `WM_TIMER`. Задаётся в миллисекундах. Таким образом, 1 сек=1000

`lpTFunc` - пользовательская функция, которую надо вызывать по истечении интервала. Если значение равно `NULL`, будет вызываться `WM_TIMER`. Функция должна быть определена, как `VOID CALLBACK` и иметь те же параметры, что и оконная функция.

Чтобы завершить работу таймера используют функцию `KillTimer`.

```
BOOL KillTimer(HWND hWnd, UINT nID);
```

hWnd - идентификатор окна

nID - номер запущенного таймера.

Сделаем простую программу, которая вызывает каждую секунду новое диалоговое окно. Так в простой и понятной форме Вы познакомитесь с работой таймера.

Создайте пустой Win API проект MyTimer и включите в него пустой файл MyTimer.cpp. Добавьте его к проекту и наберите в него следующий код.

```
#include<windows.h>

//Создаём прототип функции окна
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
//объявляем имя программы
char szProgName[]="Имя программы";

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPSTR lpszCmdLine, int nCmdShow)
{
    HWND hWnd;
    MSG lpMsg;
    WNDCLASS w; //создаём экземпляр структуры WNDCLASS
    //И начинаем её заполнять
    w.lpszClassName=szProgName; //имя программы - объявлено выше
    w.hInstance=hInstance; //идентификатор текущего приложения
    w.lpfnWndProc=WndProc; //указатель на функцию окна
    w.hCursor=LoadCursor(NULL, IDC_ARROW); //загружаем курсор
    w.hIcon=0; //иконки у нас не будет пока
    w.lpszMenuName=0; //и меню пока не будет
    w.hbrBackground=(HBRUSH)GetStockObject(WHITE_BRUSH); //цвет фона
    окна
    w.style=CS_HREDRAW|CS_VREDRAW; //стиль - перерисовываемое по x и
    по y
    w.cbClsExtra=0;
    w.cbWndExtra=0;

    //Если не удалось зарегистрировать класс окна - выходим
    if(!RegisterClass(&w))
        return 0;

    //Создадим окно в памяти, заполнив аргументы CreateWindow
    hWnd=CreateWindow(szProgName, //Имя программы
    "Моя первая программа!", //Заголовок окна
```

```

WS_OVERLAPPEDWINDOW, //Стиль окна - перекрывающееся
100, //положение окна на экране по x
100, //по y
500, //размеры по x
400, //по y
(HWND)NULL, //идентификатор родительского окна
(HMENU)NULL, //идентификатор меню
(HINSTANCE)hInstance, //идентификатор экземпляра программы
(HINSTANCE)NULL); //отсутствие дополнительных параметров

//Выводим окно из памяти на экран
ShowWindow(hWnd, nCmdShow);
//Обновим содержимое окна
UpdateWindow(hWnd);

//Установим таймер с интервалом 1 сек
SetTimer(hWnd, 1, 100, NULL);

//Цикл обработки сообщений

while(GetMessage(&lpMsg, NULL, 0, 0)) { //Получаем сообщение из очереди
TranslateMessage(&lpMsg); //Преобразует сообщения клавиш в символы
DispatchMessage(&lpMsg); //Передаёт сообщение соответствующей функции
окна
}
return(lpMsg.wParam);
}

//Функция окна
LRESULT CALLBACK WndProc(HWND hWnd, UINT messg,
WPARAM wParam, LPARAM lParam)
{
HDC hdc; //создаём контекст устройства
PAINTSTRUCT ps; //создаём экземпляр структуры графического вывода

//Цикл обработки сообщений
switch(messg)
{

//Сообщение таймера
case WM_TIMER:
//Вызов диалогового окна
MessageBox(hWnd, "Остановишь меня?", "Сообщение от таймера",
MB_OK|MB_ICONQUESTION);
break;

```

```

//сообщение выхода - разрушение окна
case WM_DESTROY:
PostQuitMessage(0); //Посылаем сообщение выхода с кодом 0 - нормальное
завершение
KillTimer(hWnd, 1); //"Убиваем" таймер
break;

default:
return(DefWindowProc(hWnd,  msg,  wParam,  lParam)); //освобождаем
очередь приложения от нераспознанных
}
return 0;
}

```

Чтобы поменять цвет фона диалогового окна:

```

w.hbrBackground=(HBRUSH)GetStockObject(WHITE_BRUSH);

```

hbrBackground - цвет кисти, которой будет залито окно. Функция GetStockObject - возвращает идентификатора кисти. Здесь её аргумент - константа WHITE_BRUSH. Если наши запросы невелики, мы можем использовать один из стандартных цветов заливки окна, определённых константами типа COLOR_MENU и др. Но в новых версиях Windows они почти все не работают, поэтому по будем собирать цвет фона из трёх компонент RGB.

```

w.hbrBackground=(HBRUSH)CreateSolidBrush(RGB(0,0,100)); //Радикально-
синий цвет

```

Поменяйте в предыдущей программе строку с цветом фона на эту и замените всё сообщение таймера на приведённое ниже.

```

case WM_TIMER:
hdc1=GetDC(hWnd); //Выделяем контекст отображения
x=rand()%500; //случайное число по x от 0 до 500
y=rand()%400; //по y от 0 до 400
SetPixel(hdc1, x,y,RGB(255,255,255)); //Выводим точку

ReleaseDC(hWnd, hdc1);
break;

```

Если мы хотим вдруг остановить появление элементов, надо только щёлкнуть левой кнопкой мыши в окне, вставив предварительно обработку сообщения мыши:

```
case WM_LBUTTONDOWN:  
KillTimer(hWnd, 1);  
break;
```

Задание: выведите на экран бегущую строку от правого края до левого, причём по достижении левого края, она должна повернуть вправо и так бесконечно. Строку вывести шрифтом Arial, высотой 20. Текст строки - текущее время.

Задание: напишите программу, которая будет вызывать несколько диалоговых окон, в каждом из которых будут рассмотрены кое-какие элементы управления. В *первом* - радиокнопки, во *втором* галочки, в *третьем* список, в *четвёртом* комбинированный список.